

Projet Business Intelligence — AL OMRANE

1. Présentation du projet

Ce projet consiste à mettre en place une solution complète de Business Intelligence pour la société immobilière **AL OMRANE**.

L'objectif est de partir d'une base de données opérationnelle, contenant des données transactionnelles liées aux clients, agents immobiliers, projets, biens immobiliers et ventes, puis de transformer ces données à travers un processus ETL afin de les charger dans un Data Warehouse. Ce Data Warehouse est ensuite utilisé pour produire des tableaux de bord décisionnels permettant d'analyser les ventes immobilières.

L'architecture générale du projet est la suivante :

```
Génération de données avec Python/Faker
↓
Fichiers CSV
↓
Base PostgreSQL Source
↓
ETL avec Apache Hop
↓
Data Warehouse PostgreSQL
↓
Tableaux de bord Metabase
```

Ce projet simule un environnement BI réel dans lequel la base transactionnelle est séparée de la base analytique. La base source représente le système opérationnel, tandis que le Data Warehouse est conçu pour l'analyse, les agrégations, les rapports et les tableaux de bord.

2. Choix de la stack technique

La stack utilisée est :

```
Linux / Ubuntu
Docker
PostgreSQL
pgAdmin
Python
Faker
Pandas
```

```
Apache Hop  
Metabase  
VS Code
```

Docker est utilisé pour isoler les services et éviter d'installer PostgreSQL, pgAdmin ou Metabase directement sur la machine.

PostgreSQL est utilisé pour deux bases différentes :

```
alomrane_source  
alomrane_dw
```

La base `alomrane_source` contient les données opérationnelles.

La base `alomrane_dw` contient le Data Warehouse.

Python avec Faker et Pandas est utilisé pour générer un jeu de données réaliste sous forme de fichiers CSV.

Apache Hop est utilisé comme outil ETL.

Metabase est utilisé pour créer les tableaux de bord décisionnels.

3. Pourquoi Apache Hop au lieu de Pentaho ?

Au début du projet, l'outil prévu était **Pentaho Data Integration**, aussi appelé Kettle ou Spoon, car il s'agit d'un outil ETL classique souvent utilisé dans les projets Business Intelligence.

Cependant, les anciennes versions Community Edition de Pentaho sont devenues difficiles à trouver depuis les sources officielles modernes. Les liens historiques ne sont plus aussi accessibles, et une grande partie de l'écosystème Pentaho est aujourd'hui orientée vers des distributions plus commerciales ou enterprise.

Des miroirs communautaires existent encore, par exemple :

```
https://github.com/ambientelivre/legacy-pentaho-ce
```

Mais pour éviter de dépendre d'anciennes archives et pour utiliser un outil plus moderne, le projet a été migré vers **Apache Hop**.

Apache Hop est un bon remplacement car :

- il est open-source ;
- il est moderne ;
- il est proche de l'esprit Pentaho/Kettle ;
- il permet de créer des pipelines ETL graphiquement ;
- il fonctionne bien avec PostgreSQL ;
- il s'intègre mieux dans un projet Git ;
- il est plus adapté à une stack data moderne.

Le changement d'outil ne modifie pas l'objectif du projet. Apache Hop remplit le même rôle que Pentaho : extraire, transformer et charger les données.

4. Structure du projet

La structure du projet est organisée comme suit :

```
alomrane-bi/
|
├── docker/
│   └── docker-compose.yml
|
├── sql/
│   ├── 001_source_schema.sql
│   └── 002_dw_schema.sql
|
├── scripts/
│   └── generate_data.py
|
├── data/
│   ├── clients.csv
│   ├── agents_immobilier.csv
│   ├── projets.csv
│   ├── biens_immobilier.csv
│   └── transactions.csv
|
├── hop/
│   ├── metadata/
│   ├── pipelines/
│   │   ├── load_dim_client.hpl
│   │   ├── load_dim_agents.hpl
│   │   ├── load_dim_projets.hpl
│   │   ├── load_dim_bien.hpl
│   │   └── load_fact_ventes.hpl
|   |
|   └──
```

```
|   |   | workflows/
|   |   |   | alomrane_etl.hwf
|   |   |
|   |   | docs/
|   |   |   | screenshots/
|   |   |
|   |   | README.md
```

Le dossier `docker/` contient l'infrastructure Docker.

Le dossier `sql/` contient les scripts SQL.

Le dossier `scripts/` contient le script Python de génération des données.

Le dossier `data/` contient les fichiers CSV générés.

Le dossier `hop/` contient les pipelines et workflows Apache Hop.

Le dossier `docs/` contient les captures d'écran et éléments de documentation.

5. Infrastructure Docker

Le projet utilise Docker Compose pour lancer les services nécessaires.

Les services sont :

```
PostgreSQL
pgAdmin
Metabase
```

PostgreSQL contient les bases de données du projet.

pgAdmin permet de visualiser et administrer PostgreSQL.

Metabase permet de construire les tableaux de bord.

Exemple de configuration Docker Compose :

```
services:
  postgres:
    image: postgres:16
    container_name: alomrane_postgres
```

```
restart: unless-stopped

environment:
  POSTGRES_USER: amulvaro
  POSTGRES_PASSWORD: amulvaro123
  POSTGRES_DB: alomrane_source

ports:
  - "5440:5432"

volumes:
  - postgres_data:/var/lib/postgresql/data
  - ../sql:/sql

mem_limit: 768m
cpus: 1.0

networks:
  - alomrane_net

pgadmin:
  image: dpage/pgadmin4
  container_name: alomrane_pgadmin
  restart: unless-stopped

  environment:
    PGADMIN_DEFAULT_EMAIL: admin@alomrane.com
    PGADMIN_DEFAULT_PASSWORD: admin123

  ports:
    - "5080:80"

  volumes:
    - pgadmin_data:/var/lib/pgadmin

  mem_limit: 512m
  cpus: 0.5

  depends_on:
    - postgres

  networks:
    - alomrane_net

metabase:
  image: metabase/metabase:latest
  container_name: alomrane_metabase
  restart: unless-stopped
```

```
ports:
  - "3000:3000"

environment:
  JAVA_OPTS: "-Xmx1g -Xms512m"

depends_on:
  - postgres

mem_limit: 1536m
cpus: 1.0

networks:
  - alomrane_net

volumes:
  postgres_data:
  pgadmin_data:

networks:
  alomrane_net:
    driver: bridge
```

6. Comprendre les ports Docker

Dans Docker, une ligne comme :

```
5440:5432
```

signifie :

```
PORT_MACHINE:PORT_CONTENEUR
```

Dans ce projet :

```
5440 = port exposé sur Ubuntu
5432 = port PostgreSQL à l'intérieur du conteneur
```

Depuis Ubuntu, la connexion PostgreSQL se fait donc avec :

```
localhost:5440
```

Mais depuis un autre conteneur du même réseau Docker, par exemple Metabase, la connexion se fait avec :

```
postgres:5432
```

Cela signifie que Metabase ne doit pas utiliser `localhost`, car dans un conteneur Docker, `localhost` désigne le conteneur lui-même.

7. Lancement des services

Depuis le dossier `docker /` :

```
docker compose up -d
```

Vérifier les conteneurs :

```
docker ps
```

Les conteneurs attendus sont :

```
alomrane_postgres  
alomrane_pgadmin  
alomrane_metabase
```

pgAdmin est accessible sur :

```
http://localhost:5080
```

Metabase est accessible sur :

```
http://localhost:3000
```

8. Bases de données PostgreSQL

Le projet utilise deux bases de données.

Base source

```
alomrane_source
```

Cette base représente le système opérationnel.

Elle contient :

```
clients  
agents_immobilier  
projets  
biens_immobilier  
transactions
```

Data Warehouse

```
alomrane_dw
```

Cette base représente le système analytique.

Elle contient :

```
dim_client  
dim_agent  
dim_projet  
dim_bien  
dim_temps  
fact_ventes
```

La séparation entre base source et Data Warehouse est importante. La base source sert aux opérations quotidiennes, tandis que le Data Warehouse sert aux analyses, rapports et tableaux de bord.

9. Connexion à PostgreSQL

Connexion à la base source :

```
docker exec -it alomrane_postgres psql -U amulvaro -d alomrane_source
```

Connexion au Data Warehouse :

```
docker exec -it alomrane_postgres psql -U amulvaro -d alomrane_dw
```

Commandes utiles :

```
\1
```

Liste les bases de données.

```
\dt
```

Liste les tables de la base actuelle.

```
\d nom_table
```

Affiche la structure d'une table.

```
\q
```

Quitte PostgreSQL.

10. Génération des données avec Python

Python est utilisé pour générer un jeu de données réaliste.

Créer un environnement virtuel :

```
python3 -m venv .venv
```

Activer l'environnement :

```
source .venv/bin/activate
```

Installer les dépendances :

```
pip install faker pandas
```

Le script de génération se trouve dans :

```
scripts/generate_data.py
```

Exécuter le script :

```
python scripts/generate_data.py
```

Le script génère les fichiers CSV suivants :

```
data/clients.csv  
data/agents_immobilier.csv  
data/projets.csv  
data/biens_immobilier.csv  
data/transactions.csv
```

Les volumes générés sont environ :

```
100 clients  
20 agents immobiliers  
15 projets  
200 biens immobiliers  
1000 transactions
```

L'utilisation de Faker permet de générer un volume de données suffisant pour produire des tableaux de bord intéressants.

11. Création de la base source

Le script SQL de la base source est :

```
sql/001_source_schema.sql
```

Depuis la racine du projet :

```
docker exec -i alomrane_postgres psql -U amulvaro -d alomrane_source < sql/001_source_schema.sql
```

Depuis le dossier `docker/` :

```
docker exec -i alomrane_postgres psql -U amulvaro -d alomrane_source < ../sql/001_source_schema.sql
```

Les tables créées sont :

```
clients
agents_immobilier
projets
biens_immobilier
transactions
```

12. Importation des fichiers CSV

Copier les CSV dans le conteneur PostgreSQL :

Depuis la racine :

```
docker cp data/. alomrane_postgres:/data
```

Depuis le dossier `docker/` :

```
docker cp ../data/. alomrane_postgres:/data
```

Se connecter à la base source :

```
docker exec -it alomrane_postgres psql -U amulvaro -d alomrane_source
```

Importer les fichiers dans l'ordre :

```
COPY clients FROM '/data/clients.csv' DELIMITER ',' CSV HEADER;

COPY agents_immobilier FROM '/data/agents_immobilier.csv' DELIMITER ',' CSV
```

```
HEADER;  
  
COPY projets FROM '/data/projets.csv' DELIMITER ',' CSV HEADER;  
  
COPY biens_immobilier FROM '/data/biens_immobilier.csv' DELIMITER ',' CSV  
HEADER;  
  
COPY transactions FROM '/data/transactions.csv' DELIMITER ',' CSV HEADER;
```

L'ordre est important car certaines tables dépendent d'autres tables via des clés étrangères.

Vérifier les données :

```
SELECT COUNT(*) FROM clients;  
SELECT COUNT(*) FROM agents_immobilier;  
SELECT COUNT(*) FROM projets;  
SELECT COUNT(*) FROM biens_immobilier;  
SELECT COUNT(*) FROM transactions;
```

13. Création du Data Warehouse

Le script du Data Warehouse est :

```
sql/002_dw_schema.sql
```

Exécution depuis la racine :

```
docker exec -i alomrane_postgres psql -U amulvaro -d alomrane_dw < sql/  
002_dw_schema.sql
```

Exécution depuis `docker/` :

```
docker exec -i alomrane_postgres psql -U amulvaro -d alomrane_dw < ../sql/  
002_dw_schema.sql
```

Le Data Warehouse suit un modèle en étoile.

Les dimensions sont :

```
dim_client  
dim_agent  
dim_projet  
dim_bien  
dim_temps
```

La table de faits est :

```
fact_ventes
```

14. Modèle en étoile

Le modèle en étoile permet d'organiser les données pour l'analyse.

La table centrale est :

```
fact_ventes
```

Elle contient les mesures :

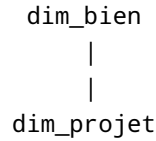
```
montant  
nombre_transaction
```

Elle référence les dimensions :

```
dim_client  
dim_agent  
dim_projet  
dim_bien  
dim_temps
```

Schéma logique :

```
          dim_temps  
          |  
          |  
dim_client --- fact_ventes --- dim_agent  
          |  
          |
```



Les dimensions donnent le contexte de chaque vente.

La table de faits contient les indicateurs numériques analysables.

15. Dimension temps

La dimension temps est remplie en SQL.

Connexion au Data Warehouse :

```
docker exec -it alomrane_postgres psql -U amulvaro -d alomrane_dw
```

Insertion des dates :

```
INSERT INTO dim_temps (date_complete, jour, mois, annee, trimestre)
SELECT
    d::date AS date_complete,
    EXTRACT(DAY FROM d)::int AS jour,
    EXTRACT(MONTH FROM d)::int AS mois,
    EXTRACT(YEAR FROM d)::int AS annee,
    EXTRACT(QUARTER FROM d)::int AS trimestre
FROM generate_series('2021-01-01'::date, '2025-12-31'::date, interval '1 day')
AS d;
```

Vérification :

```
SELECT COUNT(*) FROM dim_temps;
SELECT * FROM dim_temps LIMIT 5;
```

Cette dimension permet d'analyser les ventes par jour, mois, trimestre et année.

16. ETL avec Apache Hop

Apache Hop est utilisé pour charger le Data Warehouse à partir de la base source.

Deux connexions PostgreSQL sont créées dans Hop.

Connexion source

```
Name: alomrane_source
Type: PostgreSQL
Host: localhost
Port: 5440
Database: alomrane_source
Username: amulvaro
Password: amulvaro123
```

Connexion Data Warehouse

```
Name: alomrane_dw
Type: PostgreSQL
Host: localhost
Port: 5440
Database: alomrane_dw
Username: amulvaro
Password: amulvaro123
```

Hop est lancé localement sur Ubuntu, donc il utilise `localhost:5440`.

17. Pipelines des dimensions

Les dimensions simples utilisent un flux :

```
Table Input
  ↓
Table Output
```

load_dim_client.hpl

Source :

```
SELECT
  id_client,
  nom,
  prenom,
  adresse,
```

```
    telephone,  
    email  
FROM clients;
```

Destination :

dim_client

load_dim_agents.hpl

Source :

```
SELECT  
    id_agent,  
    nom,  
    prenom,  
    email,  
    adresse  
FROM agents_immobilier;
```

Destination :

dim_agent

load_dim_projets.hpl

Source :

```
SELECT  
    id_projet,  
    nom_projet,  
    date_debut,  
    date_fin  
FROM projets;
```

Destination :

dim_projet

load_dim_bien.hpl

Source :

```
SELECT
    id_bien,
    type_bien,
    nombre_piece,
    prix_affiche,
    region,
    ville,
    adresse
FROM biens_immobilier;
```

Destination :

dim_bien

Pour ces pipelines, les clés techniques comme `sk_client`, `sk_agent`, `sk_projet` et `sk_bien` ne sont pas mappées manuellement. Elles sont générées automatiquement par PostgreSQL.

18. Pipeline de la table de faits

Le pipeline le plus important est :

load_fact_ventes.hpl

Il charge la table :

fact_ventes

Contrairement aux dimensions, ce pipeline ne peut pas simplement copier les champs un par un.

La source contient des IDs métier :

```
id_client
id_agent
id_bien
```

```
id_projet
date_transaction
```

Mais la table de faits attend des surrogate keys :

```
sk_client
sk_agent
sk_bien
sk_projet
sk_temps
```

Le pipeline utilise donc des Database Lookups.

Structure du pipeline :

```
input_transaction
  ↓
lookup_client
  ↓
lookup_agent
  ↓
lookup_bien
  ↓
lookup_projet
  ↓
lookup_temps
  ↓
output_fact_ventes
```

19. Requête d'entrée de la table de faits

```
SELECT
    t.id_client,
    t.id_agent,
    t.id_bien,
    b.id_projet,
    t.date_transaction,
    t.montant,
    1 AS nombre_transaction
FROM transactions t
```

```
JOIN biens_immobilier b
ON b.id_bien = t.id_bien;
```

Le `JOIN` avec `biens_immobilier` permet de récupérer `id_projet`, car le projet est lié au bien immobilier.

20. Configuration des lookups

lookup_client

```
Table: dim_client
Key: id_client = id_client
Return: sk_client
```

lookup_agent

```
Table: dim_agent
Key: id_agent = id_agent
Return: sk_agent
```

lookup_bien

```
Table: dim_bien
Key: id_bien = id_bien
Return: sk_bien
```

lookup_projet

```
Table: dim_projet
Key: id_projet = id_projet
Return: sk_projet
```

lookup_temps

```
Table: dim_temps
Key: date_complete = date_transaction
Return: sk_temps
```

Chaque lookup prend une clé métier du flux et récupère la surrogate key correspondante dans le Data Warehouse.

21. Mapping de sortie de fact_ventes

La table `fact_ventes` est définie ainsi :

```
CREATE TABLE fact_ventes (  
    sk_fact SERIAL PRIMARY KEY,  
    sk_client INT REFERENCES dim_client(sk_client),  
    sk_agent INT REFERENCES dim_agent(sk_agent),  
    sk_projet INT REFERENCES dim_projet(sk_projet),  
    sk_bien INT REFERENCES dim_bien(sk_bien),  
    sk_temps INT REFERENCES dim_temps(sk_temps),  
    montant FLOAT,  
    nombre_transaction INT  
);
```

Dans Hop, le `Table Output` doit insérer seulement :

```
sk_client  
sk_agent  
sk_projet  
sk_bien  
sk_temps  
montant  
nombre_transaction
```

Il ne faut pas insérer :

```
id_client  
id_agent  
id_bien  
id_projet  
date_transaction  
sk_fact
```

`sk_fact` est généré automatiquement par PostgreSQL.

Mapping correct :

Database field	Stream field
sk_client	sk_client
sk_agent	sk_agent

sk_projet	sk_projet
sk_bien	sk_bien
sk_temps	sk_temps
montant	montant
nombre_transaction	nombre_transaction

22. Workflow Apache Hop

Après la création des pipelines, un workflow permet de tout exécuter dans le bon ordre.

Fichier :

```
hop/workflows/alomrane_etl.hwf
```

Ordre d'exécution :

```
Start
  ↓
load_dim_client
  ↓
load_dim_agents
  ↓
load_dim_projets
  ↓
load_dim_bien
  ↓
load_fact_ventes
```

La table de faits est exécutée en dernier car elle dépend de toutes les dimensions.

23. Vérification du Data Warehouse

Après l'exécution du workflow, vérifier les tables :

```
docker exec -it alomrane_postgres psql -U amulvaro -d alomrane_dw
```

Puis :

```
SELECT COUNT(*) FROM dim_client;
SELECT COUNT(*) FROM dim_agent;
SELECT COUNT(*) FROM dim_projet;
SELECT COUNT(*) FROM dim_bien;
SELECT COUNT(*) FROM dim_temps;
SELECT COUNT(*) FROM fact_ventes;
```

Résultats attendus :

dim_client	100
dim_agent	20
dim_projet	15
dim_bien	200
dim_temps	environ 1826
fact_ventes	environ 1000

24. Metabase

Metabase est utilisé pour créer les tableaux de bord.

Accès :

```
http://localhost:3000
```

Connexion PostgreSQL dans Metabase :

```
Host: postgres
Port: 5432
Database: alomrane_dw
Username: amulvaro
Password: amulvaro123
```

Metabase utilise `postgres:5432` car il est dans le même réseau Docker que PostgreSQL.

25. Questions Metabase

Dans Metabase, une question est une requête ou une visualisation sauvegardée.

Flux :

SQL Query



Save as Question



Add to Dashboard

Total des ventes

```
SELECT
    SUM(montant) AS total_ventes
FROM fact_ventes;
```

Nombre de transactions

```
SELECT
    COUNT(*) AS nombre_transactions
FROM fact_ventes;
```

Montant moyen

```
SELECT
    AVG(montant) AS montant_moyen
FROM fact_ventes;
```

Ventes par agent

```
SELECT
    a.nom,
    a.prenom,
    SUM(f.montant) AS total_ventes
FROM fact_ventes f
JOIN dim_agent a
    ON a.sk_agent = f.sk_agent
GROUP BY a.nom, a.prenom
ORDER BY total_ventes DESC;
```

Ventes par région

```
SELECT
    b.region,
    SUM(f.montant) AS total_ventes
```

```
FROM fact_ventes f
JOIN dim_bien b
    ON b.sk_bien = f.sk_bien
GROUP BY b.region
ORDER BY total_ventes DESC;
```

Ventes par ville

```
SELECT
    b.ville,
    SUM(f.montant) AS total_ventes
FROM fact_ventes f
JOIN dim_bien b
    ON b.sk_bien = f.sk_bien
GROUP BY b.ville
ORDER BY total_ventes DESC;
```

Évolution mensuelle des ventes

```
SELECT
    t.annee,
    t.mois,
    SUM(f.montant) AS total_ventes
FROM fact_ventes f
JOIN dim_temps t
    ON t.sk_temps = f.sk_temps
GROUP BY t.annee, t.mois
ORDER BY t.annee, t.mois;
```

Ventes par type de bien

```
SELECT
    b.type_bien,
    SUM(f.montant) AS total_ventes
FROM fact_ventes f
JOIN dim_bien b
    ON b.sk_bien = f.sk_bien
GROUP BY b.type_bien
ORDER BY total_ventes DESC;
```

26. Dashboard Metabase

Créer un dashboard nommé :

AL OMRANE BI Dashboard

Ajouter les questions suivantes :

Total des ventes
Nombre de transactions
Montant moyen
Ventes par agent
Ventes par région
Ventes par ville
Évolution mensuelle des ventes
Ventes par type de bien

Le dashboard permet d'analyser :

la performance commerciale
les ventes par région
les ventes par ville
les performances des agents
l'évolution des ventes dans le temps
les types de biens les plus rentables

27. Architecture finale

L'architecture finale du projet est :

Fichiers CSV
↓
PostgreSQL Source
↓
Apache Hop ETL
↓
PostgreSQL Data Warehouse
↓
Metabase Dashboards

Le projet couvre tout le cycle BI :

```
génération de données  
stockage opérationnel  
modélisation dimensionnelle  
ETL  
Data Warehouse  
visualisation  
aide à la décision
```

28. Commandes utiles

Lancer les services :

```
cd docker  
docker compose up -d
```

Arrêter les services :

```
docker compose stop
```

Supprimer les conteneurs :

```
docker compose down
```

Attention : cette commande supprime aussi les volumes et donc les données :

```
docker compose down --volumes
```

Connexion à la base source :

```
docker exec -it alomrane_postgres psql -U amulvaro -d alomrane_source
```

Connexion au Data Warehouse :

```
docker exec -it alomrane_postgres psql -U amulvaro -d alomrane_dw
```

Voir les tables :

dt

Vérifier la table de faits :

```
SELECT COUNT(*) FROM fact_ventes;
```

Ouvrir pgAdmin :

http://localhost:5080

Ouvrir Metabase :

http://localhost:3000

29. État actuel du projet

Terminé :

- Configuration Docker
- PostgreSQL
- pgAdmin
- Metabase
- Base source
- Génération Faker
- Import CSV
- Data Warehouse
- Pipelines Apache Hop
- Workflow Apache Hop
- Connexion Metabase

À finaliser :

- Dashboard Metabase
- Captures d'écran

Rapport final
Préparation de la démonstration

30. Auteur

Amulvaro
Projet Business Intelligence
AL OMRANE